Processing Medical Data for Calculating a Typical Day of a Patient with Diabetes

Semester Project for Course KIV/PPR at UWB in Pilsen

Bc. Jiří Winter

February 18, 2026

Contents

1 Introduction and Project Structure **3**
- 1.1 Medical Data . . . 3
- 1.2 Directory Structure . . . 3
- 1.3 Running the Program . . . 3

2 Data Processing Algorithm **4**
- 2.1 Phase 1: Median per Patient . . . 4
- 2.2 Phase 2: Median Across Patients (Grand Median) . . . 4
- 2.3 Phase 3: Downsampling (Time Slot Reduction) . . . 5

3 Data Loading and Representation **6**
- 3.1 Flat Data Layout . . . 6
- 3.2 Parsing Timestamps . . . 6
  - 3.2.1 Frequency 15ms . . . 6
- 3.3 Handling Empty Values (Padding) . . . 7

4 Sequential Processing (CPU) **8**
- 4.1 Median Calculation Example . . . 8
- 4.2 Sequential Algorithm Example . . . 9
- 4.3 Algorithm Selection: std::nth_element . . . 10

5 Parallelization and SIMD Optimization **11**
- 5.1 Algorithm Parallelization . . . 11
- 5.2 Variant 1: Parallelization without SIMD . . . 11
- 5.3 Variant 2: Parallelization with SIMD . . . 11
  - 5.3.1 Principle of SIMD Sorting - Sorting Network . . . 13
  - 5.3.2 Median Extraction from Sorted Registers . . . 13
- 5.4 Median Across Patients . . . 14
- 5.5 Time Slot Downsampling . . . 15

6 GPU (OpenCL) **17**
- 6.1 Kernel 1: Median per Patient . . . 19
- 6.2 Kernel 2: Reduce Patients . . . 19
- 6.3 Kernel 3: Downsampling (Shell Sort) . . . 21
- 6.4 Solution for BVP Data (740 million records) . . . 22

7 Measured Results and Evaluation **23**
- 7.1 Hardware Configuration . . . 23
- 7.2 Data Loading Times . . . 23
- 7.3 Dexcom Data (Low Load) . . . 23
- 7.4 HR Data (Medium Load) . . . 24

7.5 BVP Data (High Load) 24
7.6 Analysis of GPU Calculation Phases 25

8 Analysis and Discussion of Results **26**
8.1 Speedup and Efficiency 26
8.2 Karp-Flatt Metric 27
8.3 Scalability over Time 27

9 Discussion on Implementation **29**

10 Conclusion **30**

Appendix A: User Manual **31**

Chapter 1

Introduction and Project Structure

The goal of this work is the effective processing of large volumes of medical data (Dexcom, HR, BVP) with varying sampling frequencies (from 5 minutes down to 15 ms). The application calculates medians of measured values over time, performs data reduction (downsampling), and utilizes CPU parallelization methods (OpenMP, SIMD) and GPU parallelization (OpenCL).

1.1 Medical Data

The work can process three types of data:

- **Dexcom Data:** Glucose data sampled every 5 minutes (288 records/day).
- **HR Data:** Heart rate sampled every 1 second (86,400 records/day).
- **BVP Data:** Bio-voltage signal sampled every 15 ms (5,760,000 records/day).

1.2 Directory Structure

The project is divided into modules according to responsibility:

- **include/**
  - `DataReader.h` - Declaration of classes for data loading and conversion.
  - `ReadDexcomData.h` - Data structures for storing records (`DexcomData`).
- **src/**
  - `DataReader.cpp` - Implementation of the CSV parser and timestamp conversion/normalization.
  - `CPUSequential.cpp` - Reference sequential implementation.
  - `CPUParallel.cpp` - Implementation utilizing OpenMP and vectorization.
  - `GPUParallel.cpp` - Host code for OpenCL (management of buffers and kernels).
  - `kernel.cl` - OpenCL kernels for execution on the graphics card.
  - `main.cpp` - Entry point, benchmarks, and result validation.

1.3 Running the Program

The program is designed to run from the command line. The user manual is provided in Appendix A.

Chapter 2

Data Processing Algorithm

The data processing algorithm is identical for all three implementation methods (sequential, parallel, and GPU); they differ only in how the individual steps are executed.

The algorithm works with three dimensions of data:

1. **Time Slots:** Discrete time segments during the day (e.g., 00:00:00, 00:00:15, ...).
2. **Patients:** The individual people measured.
3. **Days:** Measured values for a given patient and a given time across different days (6-8 days of measurement).

The entire process consists of three main phases:

2.1 Phase 1: Median per Patient

In this phase, for every combination of **(TimeSlot, Patient)**, all measured data across all days (6-8 values) are taken, and the median is calculated.

- **Input:** Matrix [TimeSlots × Patients × Days]
- **Output:** Matrix [TimeSlots × Patients]
- **Operation:** For each patient P at time T, the vector of days $D_1 \dots D_N$ is taken, sorted, and the middle value is selected.

2.2 Phase 2: Median Across Patients (Grand Median)

Subsequently, for each **TimeSlot**, the resulting medians of all patients from the previous phase are taken, and a single representative median is calculated for that time instant.

- **Input:** Matrix [TimeSlots × Patients]
- **Output:** Vector [TimeSlots]
- **Operation:** For each time T, the vector of patients $P_1 \dots P_M$ is taken, sorted, and the middle value is selected.

2.3 Phase 3: Downsampling (Time Slot Reduction)

If necessary (e.g., for visualization or noise reduction), a reduction of the sampling frequency is performed. Original fine time slots are merged into larger blocks.

- **Example:** Reduction from 15ms sampling to 5 minutes.
- **Input:** Vector [TimeSlots] (e.g., 5,760,000 elements)
- **Output:** Vector [ReducedTimeSlots] (e.g., 288 elements)
- **Operation:** For each new (larger) window, a segment of original values is taken, sorted, and the median is calculated.

Chapter 3

Data Loading and Representation

For effective processing on the GPU, it is crucial to convert data from an object representation (vector of objects) into a linear array (Flat Data Layout).

3.1 Flat Data Layout

Data is stored in a single continuous vector `std::vector<float> flat_data`. Indexing follows the formula:

$$Index = TimeSlot \times (NumPatients \times NumDays) + Patient \times NumDays + Day \quad (3.1)$$

This layout ensures that data for a single patient within a single time slot (e.g., 8 days of measurement) lie immediately next to each other in memory, which is optimal for SIMD instructions and processor cache.

3.2 Parsing Timestamps

For each data type, it is necessary to convert the text timestamp into a time slot index for data alignment. A separate function is implemented for each sampling frequency along with a time normalization function:

- **Frequency 5 minutes:** minutes_since_midnight()
- **Frequency 1 second:** seconds_since_midnight()
- **Frequency 15 ms:** units_15ms_since_midnight()

3.2.1 Frequency 15ms

For input data, it is necessary to convert the text time to a slot index. The implementation uses `std::from_chars` for maximum speed and also handles millisecond normalization.

```
1 std::optional<uint32_t> DataConverter::units_15ms_since_midnight(const std::
    string_view& ts) {
2     // Fast path: assuming format "YYYY-MM-DD HH:MM:SS.ms"
3     if (ts.size() >= 23) {
4         // ... parsing HH, MM, SS ...
5 
6         // Helper function for parsing milliseconds
7         auto to_int = [](std::string_view s) -> std::optional<int> {
8             int out{};
```

```
            auto res = std::from_chars(s.data(), s.data() + s.size(), out);
            if (res.ec == std::errc{}) return out;
            return std::nullopt;
        };

        int digit_count = 0;
        // ... counting digits in milliseconds -> digit_count

        // Parsing milliseconds and normalization
        if (digit_count > 0) {
            auto sv_ms = ts.substr(20, digit_count);
            auto oms = to_int(sv_ms);
            if (oms) {
                ms = *oms;
                // ISO 8601 rule: .5 is 500ms, .05 is 50ms
                if (digit_count == 1) ms *= 100;
                else if (digit_count == 2) ms *= 10;
            }
        }

        // Calculation of total milliseconds and slot index
        uint32_t total_ms = (*oh * 3600 + *om * 60 + *os) * 1000 + ms;
        return total_ms / 15;
    }
    return std::nullopt;
}
```

Listing 3.1: Method for calculating slot index from time (DataReader.cpp)

3.3 Handling Empty Values (Padding)

Since there are empty sections in the measurements where data is missing, we use padding and fill this empty data with a constant. For BVP data, which can take negative values, `-1.0f` cannot be used, so the smallest possible float is used.

```
// We use a very small number so it doesn't get confused with real data
constexpr float EMPTY_VALUE = -1.0e30f;
constexpr float EPSILON = 100.0f; // Tolerance for comparison
```

Listing 3.2: Padding Definition

Chapter 4

Sequential Processing (CPU)

The reference implementation serves to verify the correctness of parallel algorithms. It iterates through time slots, patients, and days sequentially, suppressing automatic vectorization/parallelization by the compiler.

```
// During implementation and benchmark, clang compiler was used
#pragma clang loop vectorize(enable) interleave(disable)
for (...) {}
```

Listing 4.1: Padding Definition

4.1 Median Calculation Example

The following code demonstrates the median calculation, which filters padding and handles cases with an even number of elements. In such a case, the median is the average of the two middle values.

```
inline float calculateMedian(const std::vector<float>& data) {
    if (data.empty()) return EMPTY_VALUE;

    // Filtering: Create buffer only for valid data
    // Ignore EMPTY_VALUE padding, but take negative numbers (BVP)
    std::vector<float> valid_data;
    valid_data.reserve(data.size());

    for (float val : data) {
        // Condition if value is greater than EMPTY_VALUE + float deviation
        if (val > EMPTY_VALUE + EPSILON) {
            valid_data.push_back(val);
        }
    }

    if (valid_data.empty()) return EMPTY_VALUE;

    // The median calculation itself (nth_element)
    size_t n = valid_data.size() / 2;
    std::nth_element(valid_data.begin(), valid_data.begin() + n, valid_data.
    end());
    float median = valid_data[n];

    // Averaging for even number of elements
    // If element count is even, median is average of n-th and (n-1)-th
    element.
    // nth_element guaranteed n-th, for (n-1)-th we must find maximum in
    left part.
```

```
    if (valid_data.size() % 2 == 0) {
        auto max_it = std::max_element(valid_data.begin(), valid_data.begin() + n);
        median = (*max_it + median) / 2.0f;
    }
    return median;
}
```

Listing 4.2: Sequential median calculation with filtering (CPUSequential.cpp)

4.2 Sequential Algorithm Example

The following code shows the main loop of the sequential median calculation and time slot reduction. First, medians are calculated for all patients across days, then the median for each time window across patients is calculated, and finally, downsampling is performed. Results are stored in `result_medians_seq` and `updated_timeslots_seq`.

```
void DexcomData::processSequential(int32_t num_wanted_time_slots) {
#pragma clang loop vectorize(enable) interleave(disable)
    // loop across time slots
    for (auto i = 0; i < num_time_slots; i++) {
        std::vector<float> patients_medians;
        patients_medians.reserve(num_patients);
        // loop across patients
        for (auto j = 0; j < num_patients; j++) {
            std::vector<float> vectorData;
            vectorData.reserve(num_days);
            auto data = getPatientDataPtr(i, j);
            for (auto k = 0; k < num_days; k++) {
                vectorData.push_back(*(data+k));
            }
            // median for patient j in time slot i across days
            float median = calculateMedian(vectorData);
            patients_medians.push_back(median);
        }
        // median for time slot i across patients
        float median = calculateMedian(patients_medians);
        result_medians_seq.push_back(median);
    }
    // downsampling of time slots, if requested
    // num_wanted_time_slots is number of target slots after reduction
    if (num_wanted_time_slots > 0) {
        updated_timeslots_seq.reserve(num_wanted_time_slots);
        std::vector<float> vectorData;
        for (int i = 0; i < num_wanted_time_slots; ++i) {
            // Calculation of range for current target slot
            // Difference in end_idx and start_idx determines count of elements to process
            size_t start_idx = (static_cast<size_t>(i) * num_time_slots) / num_wanted_time_slots;
            size_t end_idx = (static_cast<size_t>(i + 1) * num_time_slots) / num_wanted_time_slots;

            if (end_idx > num_time_slots) end_idx = num_time_slots;
            if (start_idx >= end_idx) {
                updated_timeslots_seq.push_back(EMPTY_VALUE);
                continue;
            }

```

```
40              for (size_t k = start_idx; k < end_idx; ++k) {
41                  vectorData.push_back(result_medians_seq[k]);
42              }
43              // calculation of median for current target slot
44              float median = calculateMedian(vectorData);
45              updated_timeslots_seq.push_back(median);
46          }
47      }
48  }
```

Listing 4.3: Sequential median calculation with filtering (CPUSequential.cpp)

4.3 Algorithm Selection: std::nth_element

The function `std::nth_element` was chosen for median calculation instead of classic sorting `std::sort`. The reason is algorithmic complexity:

- `std::sort`: Complexity $\mathcal{O}(N \log N)$, sorts the entire vector.
- `std::nth_element`: Complexity $\mathcal{O}(N)$ on average. Uses the Quickselect algorithm, which only rearranges elements so that the element at the n-th position is the one that would be there in a sorted sequence.

Since we are only interested in a single value in the middle (median), full sorting is unnecessary and `std::nth_element` is significantly faster.

Chapter 5

Parallelization and SIMD Optimization

To increase performance on modern processors, a combination of thread parallelization (OpenMP) and vectorization (SIMD) was used. Two variants of parallel processing on the CPU were implemented and measured within the project. Both use the OpenMP library to distribute work among threads, but they differ in the level of calculation optimization within individual threads. One method uses standard sequential calculations, while the other uses manually optimized code for calculating the median from a fixed number of 8 values using SIMD instructions.

1. **Parallel (No SIMD):** Simple loop parallelization, which uses `std::vector` allocation and `std::nth_element` function for median calculation.
2. **Parallel + SIMD:** Manually optimized variant using SIMD instructions, which uses sorting directly in processor registers - Sorting Network.

5.1 Algorithm Parallelization

Data for different time slots are independent of each other, so it can be easily parallelized. Each time slot can thus be calculated in parallel. Parallelization takes place in two phases:

1. **Phase 1:** Calculation of medians for individual TimeSlots. The loop across time slots is divided among threads using `#pragma omp parallel for`. Each thread processes its block of time slots.
2. **Phase 2:** Reduction (Downsampling) of time windows. Parallelization across target (reduced) slots.

5.2 Variant 1: Parallelization without SIMD

This method (represented by the function `processParallelCPUNonVectorized`) parallelizes the outer loop over time slots using `#pragma omp parallel for`. Inside the thread, however, it behaves like the sequential code: for each patient, it dynamically allocates a vector, fills it with data, and calls the generic function for the median.

5.3 Variant 2: Parallelization with SIMD

This variant (function `processParallelCPU`) attempts manual optimization using SIMD vector instructions. It utilizes the fact that for the given dataset we have a maximum of 8 days of measurement. This allows sorting using a Sorting Network over vectors of 4 floats - `float32x4_t`.

One vector is created for each day, and values for 4 patients are loaded into one vector. One sorting operation thus processes 4 patients at once.

```
// Function for transposing a 4x4 matrix stored in 4 vectors
inline void transpose_matrix(float32x4_t& r0, float32x4_t& r1, float32x4_t& r2, float32x4_t& r3) {
    float32x4_t t0 = vtrn1q_f32(r0, r1);    // takes all even indices
    float32x4_t t1 = vtrn2q_f32(r0, r1);    // takes all odd indices
    float32x4_t t2 = vtrn1q_f32(r2, r3);
    float32x4_t t3 = vtrn2q_f32(r2, r3);
    r0 = vcombine_f32(vget_low_f32(t0), vget_low_f32(t2));
    r1 = vcombine_f32(vget_low_f32(t1), vget_low_f32(t3));
    r2 = vcombine_f32(vget_high_f32(t0), vget_high_f32(t2));
    r3 = vcombine_f32(vget_high_f32(t1), vget_high_f32(t3));
}

void DexcomData::processParallelCPU(int32_t num_wanted_time_slots) {
    // ... initialization of variables and vectors
    const uint32_t aligned_patients = (num_patients / 4) * 4;
    const uint32_t remainder = num_patients % 4;
    const uint32_t padding_patients = (remainder == 0) ? 0 : (4 - remainder);

    for (auto i = 0; i < padding_patients; i++) {
        for (auto j = 0; j < num_time_slots; j++) {
            flat_data.push_back(-1);
        }
    }
    const uint32_t patients_with_padding = aligned_patients + padding_patients;
    #pragma omp parallel for
    {
        for (uint32_t ts = 0; ts < num_time_slots; ++ts) {
            for (uint32_t p = 0; p < patients_with_padding; p += 4) {
                float* ptr = getPatientDataPtr(ts, p);

                // Loading data for 4 patients and 4 days of measurement at once into vectors
                // d0 contains first 4 days for patient p
                float32x4_t d0 = vld1q_f32(ptr);
                // d1 contains first 4 days for patient p+1
                float32x4_t d1 = vld1q_f32(ptr + 1*8);
                float32x4_t d2 = vld1q_f32(ptr + 2*8);
                float32x4_t d3 = vld1q_f32(ptr + 3*8);
                // Matrix transposition for correct alignment
                // d0 contains day 1 for patients p, p+1, p+2, p+3
                transpose_matrix(d0, d1, d2, d3);

                // Loading data for days 5-8
                float32x4_t d4 = vld1q_f32(ptr + 4);
                float32x4_t d5 = vld1q_f32(ptr + 1*8 + 4);
                float32x4_t d6 = vld1q_f32(ptr + 2*8 + 4);
                float32x4_t d7 = vld1q_f32(ptr + 3*8 + 4);
                // Matrix transposition for correct alignment
                transpose_matrix(d4, d5, d6, d7);

                // Median calculation for 4 patients at once ...
            }
        }
    }
```

```
}
```

Listing 5.1: Initialization of SIMD vectors for parallel median calculation (CPUParallel.cpp)

5.3.1 Principle of SIMD Sorting - Sorting Network

Instead of generic sorting, we use a so-called **Sorting Network**. It is an algorithm with a fixed number of comparisons and swaps that does not depend on data values (data-independent). The search utilizes parallel comparison and value swapping in processor registers **"Compare and Swap"** using instructions `vminq_f32` and `vmaxq_f32`.

```
#define SORT_VEC(A, B) { \
    float32x4_t min_v = vminq_f32(A, B); \
    float32x4_t max_v = vmaxq_f32(A, B); \
    A = min_v; \
    B = max_v; \
}
```

Listing 5.2: Macro for comparison and swap of two SIMD vectors (CPUParallel.cpp)

Sorting 8 values for 4 patients takes place in 21 steps (Compare and Swap).

```
void DexcomData::processParallelCPU(int32_t num_wanted_time_slots) {
    for (uint32_t ts = 0; ts < num_time_slots; ++ts) {
        for (uint32_t p = 0; p < patients_with_padding; p += 4) {
            // Loading and transposition of data (see previous code) ...

            // Manual sorting of 8 values using Sorting Network
            SORT_VEC(d0, d1); SORT_VEC(d2, d3); SORT_VEC(d4, d5); SORT_VEC(
    d6, d7);
            SORT_VEC(d0, d2); SORT_VEC(d1, d3); SORT_VEC(d4, d6); SORT_VEC(
    d5, d7);
            SORT_VEC(d1, d2); SORT_VEC(d5, d6); SORT_VEC(d0, d4); SORT_VEC(
    d3, d7);
            SORT_VEC(d1, d5); SORT_VEC(d2, d6);
            SORT_VEC(d1, d4); SORT_VEC(d3, d6);
            SORT_VEC(d2, d4); SORT_VEC(d3, d5);
            SORT_VEC(d3, d4); SORT_VEC(d1, d2); SORT_VEC(d5, d6);

            // Padding filtration and median extraction from register
    vectors ...
        }
    }
}
```

Listing 5.3: Macro for comparison and swap of two SIMD vectors (CPUParallel.cpp)

5.3.2 Median Extraction from Sorted Registers

After sorting using the Sorting Network, values in registers are ordered from smallest to largest. Padding filtration (`EMPTY_VALUE`) and median calculation follow. The count of valid values is calculated by comparison with `EMPTY_VALUE + EPSILON`, and based on this, the median position is determined.

```
    // Padding filtration and counting valid values after Sorting Network -
    see above
    float32x4_t limit = vdupq_n_f32(EMPTY_VALUE + EPSILON);
    uint32x4_t counts = vdupq_n_u32(0);

    // Summing masks to find invalid values
```

```
    counts = vsubq_u32(counts, vcgtq_f32(d0, limit));
    counts = vsubq_u32(counts, vcgtq_f32(d1, limit));
    counts = vsubq_u32(counts, vcgtq_f32(d2, limit));
    counts = vsubq_u32(counts, vcgtq_f32(d3, limit));
    counts = vsubq_u32(counts, vcgtq_f32(d4, limit));
    counts = vsubq_u32(counts, vcgtq_f32(d5, limit));
    counts = vsubq_u32(counts, vcgtq_f32(d6, limit));
    counts = vsubq_u32(counts, vcgtq_f32(d7, limit));

    uint32_t cnt[4];
    vst1q_u32(cnt, counts);

    // extraction of sorted values into array
    float sorted_cols[32];
    vst1q_f32(sorted_cols + 0, d0);
    vst1q_f32(sorted_cols + 4, d1);
    vst1q_f32(sorted_cols + 8, d2);
    vst1q_f32(sorted_cols + 12, d3);
    vst1q_f32(sorted_cols + 16, d4);
    vst1q_f32(sorted_cols + 20, d5);
    vst1q_f32(sorted_cols + 24, d6);
    vst1q_f32(sorted_cols + 28, d7);

    // Median calculation for each of the 4 patients
    for(int k=0; k<4; ++k) {
        int n = cnt[k];
        float median = EMPTY_VALUE;
        if(n > 0) {
            int start = 8 - n;
            int mid = n / 2;

            int idx1 = (start + mid) * 4 + k;
            median = sorted_cols[idx1];

            // Even number of elements - average of two middle values
            if(n % 2 == 0) {
                int idx2 = (start + mid - 1) * 4 + k;
                median = (sorted_cols[idx2] + median) / 2.0f;
            }
            grand_median_buffer.push_back(median);
        }
        result_medians_par_per_pat[(ts * num_patients) + p + k] = median;
    }
```

Listing 5.4: Median extraction from sorted SIMD registers (CPUParallel.cpp)

5.4 Median Across Patients

I did not succeed in implementing an optimal algorithm for vector sorting. Below is an example of bitonic sort implementation for 16 elements, but this did not sort values across all vectors - only within one vector of 4 elements. From a matrix perspective, it sorted columns and rows.

```
inline void bitonic_sort_16(float32x4_t& q0, float32x4_t& q1, float32x4_t&
    q2, float32x4_t& q3) {

    SORT_VEC(q0, q1); SORT_VEC(q2, q3); SORT_VEC(q0, q2); SORT_VEC(q1, q3);
    SORT_VEC(q0, q3); SORT_VEC(q1, q2);

    transpose_matrix(q0, q1, q2, q3);
```

```
    SORT_VEC(q0, q1); SORT_VEC(q2, q3); SORT_VEC(q0, q2); SORT_VEC(q1, q3);
    SORT_VEC(q0, q3); SORT_VEC(q1, q2);

    transpose_matrix(q0, q1, q2, q3);
}
```

Listing 5.5: Bitonic Sort for 16 elements in 4 vectors (CPUParallel.cpp)

Therefore, in this phase, the standard function `std::nth_element` is used, which is not vectorized, but the entire loop over time slots is still parallelized using OpenMP.

```
    // Calculation of Grand Median for each time slot
    float gm = EMPTY_VALUE;
    if (!grand_median_buffer.empty()) {
        size_t n = grand_median_buffer.size() / 2;
        std::nth_element(grand_median_buffer.begin(), grand_median_buffer.
    begin() + n, grand_median_buffer.end());
        gm = grand_median_buffer[n];

        if (grand_median_buffer.size() % 2 == 0) {
            auto max_it = std::max_element(grand_median_buffer.begin(),
    grand_median_buffer.begin() + n);
            gm = (*max_it + gm) / 2.0f;
        }
    }
```

Listing 5.6: Bitonic Sort for 16 elements in 4 vectors (CPUParallel.cpp)

5.5 Time Slot Downsampling

Time slot downsampling is implemented in parallel using OpenMP. Each thread processes one target (reduced) time slot. For each target slot, the range of original slots falling into it is calculated, and the median is calculated from these values using `std::nth_element`. The implementation is without manual SIMD optimization.

```
    #pragma omp for schedule(static)
    for (int i = 0; i < num_wanted_time_slots; ++i) {
        size_t start_idx = (static_cast<size_t>(i) * num_time_slots) /
    num_wanted_time_slots;
        size_t end_idx = (static_cast<size_t>(i + 1) * num_time_slots) /
    num_wanted_time_slots;

        if (end_idx > num_time_slots) end_idx = num_time_slots;
        size_t count = end_idx - start_idx;
        if (count <= 0) {
            updated_timeslots_par[i] = (start_idx < num_time_slots) ?
    result_medians_par[start_idx] : EMPTY_VALUE;
            continue;
        }

        // Local buffer for values in current range
        std::vector<float> local_buffer;
        // ... filling local_buffer with values from result_medians_par ...

        float median = EMPTY_VALUE;
        if (!local_buffer.empty()) {
            auto n = local_buffer.size() / 2;
            std::nth_element(local_buffer.begin(), local_buffer.begin() + n,
     local_buffer.end());
```

```
            median = local_buffer[n];

            if (n % 2 == 0) {
                auto max_it = std::max_element(local_buffer.begin(),
    local_buffer.begin() + n);
                median = (*max_it + median) / 2.0f;
            }
        }
        updated_timeslots_par[i] = median;
    }
```

Listing 5.7: Parallel time slot downsampling (CPUParallel.cpp)

Chapter 6

GPU (OpenCL)

For processing larger data (especially BVP with **740 million records** (5.7 million slots × 16 patients × 8 days)), a pipeline consisting of 3 kernels was implemented. The division into three kernels was chosen due to the synchronization of parallel operations. The implementation is realized in OpenCL.

The entire pipeline is controlled from C++ host code, which prepares data, sets up the OpenCL environment, uploads data to the GPU, launches individual kernels, and downloads results back to the CPU - `GPUParallel.cpp`. Between individual kernels, data remains stored in GPU global memory, and only final results are transferred back to program memory after the entire calculation is finished.

```
void DexcomData::processGPU(int32_t num_wanted_time_slots, bool
    read_all_outputs, int8_t num_kernels_use, const std::string& kernel_file)
     {
    // setting OpenCL environment, loading kernels from kernel_file ...

    // allocation of buffers in GPU global memory
    // input data
    cl_mem d_data = clCreateBuffer(context, CL_MEM_READ_ONLY |
        CL_MEM_COPY_HOST_PTR, flat_data.size() * sizeof(float),
        flat_data.data(), &err);
    checkErr(err, "CreateBuffer Input Data");

    // medians per patients
    cl_mem d_medians_per_patients_results = clCreateBuffer(context,
        CL_MEM_WRITE_ONLY, total_items * sizeof(float), NULL, &err);
    checkErr(err, "CreateBuffer medians per patients");

    // medians per time_slots
    cl_mem d_time_slots_results = clCreateBuffer(context, CL_MEM_READ_WRITE,
        num_time_slots * sizeof(float), NULL, &err);
    checkErr(err, "CreateBuffer medians per time_slots");

    size_t total_items = num_time_slots * num_patients;

    // kernel 1: Median calculation across days for each patient
    cl_kernel kernel = clCreateKernel(program, "median_kernel", &err);

    // setting arguments for kernel 1 ...
    clSetKernelArg(kernel, 0, sizeof(cl_mem), &d_data);
    clSetKernelArg(kernel, 1, sizeof(cl_mem),
        &d_medians_per_patients_results);
    clSetKernelArg(kernel, 2, sizeof(int), &n_days);
    clSetKernelArg(kernel, 3, sizeof(int), &n_total);
    size_t global_work_size = total_items;
```

```
33
34      err = clEnqueueNDRangeKernel(queue, kernel, 1, NULL, &global_work_size,
35          NULL, 0, NULL, NULL);
36
37      // kernel 2: Median calculation across patients for each time slot
38      if (num_kernels_use >= 2) {
39          cl_kernel timeslots_kernel = clCreateKernel(program,
40              "reduce_patients_kernel", &err);
41
42          // setting arguments for kernel 2 ...
43          clSetKernelArg(timeslots_kernel, 0, sizeof(cl_mem),
44              &d_medians_per_patients_results);
45          clSetKernelArg(timeslots_kernel, 1, sizeof(cl_mem),
46              &d_time_slots_results);
47          clSetKernelArg(timeslots_kernel, 2, sizeof(int), &num_patients);
48
49          size_t global_work_size2 = num_time_slots;
50
51          err = clEnqueueNDRangeKernel(queue, timeslots_kernel,
52              1, NULL, &global_work_size2, NULL, 0, NULL, NULL);
53
54          // release kernel 2
55      }
56      // kernel 3: Time slot downsampling
57      if (num_kernels_use >= 3 && num_wanted_time_slots > 0) {
58
59          // buffer for downsampling results
60          cl_mem d_wanted_time_slots_results = clCreateBuffer(context,
61              CL_MEM_READ_WRITE, num_wanted_time_slots * sizeof(float),
62              NULL, &err);
63          checkErr(err, "CreateBuffer medians per wanted_time_slots");
64
65          cl_kernel reduce_slots_kernel = clCreateKernel(program,
66              "reduce_slots_kernel", &err);
67
68          // setting arguments for kernel 3 ...
69          clSetKernelArg(reduce_slots_kernel, 0, sizeof(cl_mem),
70              &d_time_slots_results);
71          clSetKernelArg(reduce_slots_kernel, 1, sizeof(cl_mem),
72              &d_wanted_time_slots_results);
73          clSetKernelArg(reduce_slots_kernel, 2, sizeof(int),
74              &num_time_slots);
75          clSetKernelArg(reduce_slots_kernel, 3, sizeof(int),
76              &num_wanted_time_slots);
77
78          size_t global_work_size3 = num_wanted_time_slots;
79
80          err = clEnqueueNDRangeKernel(queue, reduce_slots_kernel,
81              1, NULL, &global_work_size3, NULL, 0, NULL, NULL);
82
83          // downloading results from GPU
84          rr = clEnqueueReadBuffer(queue, d_wanted_time_slots_results,
85              CL_TRUE, 0, num_wanted_time_slots * sizeof(float),
86              updated_timeslots_gpu.data(), 0, NULL, NULL);
87
88          // release kernel 3
89      }
90      // release other GPU resources ...
91
92  }
```

6.1 Kernel 1: Median per Patient

This kernel calculates the median across 8 days for each patient in every time window.

- **Input:** Raw data [TimeSlots × Patients × Days]
- **Implementation:** Each thread (Work-item) processes one patient at one time. Since the number of days is fixed (8), data is loaded into private registers and sorted using a **Sorting Network** (series of `CAS` instructions).

```
__kernel void median_kernel(
    __global const float* data,
    __global float* result_medians,
    const int num_days,
    const int total_patients_slots
) {
    int gid = get_global_id(0);
    long offset = (long)gid * (long)num_days;

    // Loading 8 days into registers
    float v0 = data[offset + 0];
    float v1 = data[offset + 1];
    float v2 = data[offset + 2];
    float v3 = data[offset + 3];
    float v4 = data[offset + 4];
    float v5 = data[offset + 5];
    float v6 = data[offset + 6];
    float v7 = data[offset + 7];

    // Sorting Network (Compare-And-Swap)
    CAS(v0, v1); CAS(v2, v3); CAS(v4, v5); CAS(v6, v7);
    // ... further steps of the network ...

    // Filtering and median calculation
    int valid_count = 0;
    valid_count += (v0 > VALID_THRESHOLD);
    // ...

    if (valid_count > 0) {
        // ... selection of median from 'sorted' array and saving to
   result_medians ...
    }
}
```

Listing 6.1: Median Kernel with CAS network (kernel.cl)

6.2 Kernel 2: Reduce Patients

This kernel calculates the median across all patients for each time slot based on the output from the first kernel.

- **Input:** Output from Kernel 1 (medians of individual patients).
- **Task:** For each time slot, calculate the median from all patients.
- **Reason for separation:** Calculation of the median across patients requires access to data produced by different threads in Kernel 1. Merging into a single kernel would require

global barriers or atomic operations, which would degrade performance. By launching a new kernel, synchronization is guaranteed globally.

- **Implementation:** Each thread loads medians of all patients for the given time into a local buffer and performs sorting.

```
__kernel void reduce_patients_kernel(
    __global const float* input_matrix,
    __global float* output_medians,
    const int num_patients
) {
    int ts = get_global_id(0); // time_slot

    #define MAX_BUFFER_SIZE 16
    float buffer[MAX_BUFFER_SIZE];

    int valid_count = 0;
    long offset = (long)ts * (long)num_patients;

    for (int p = 0; p < num_patients; ++p) {
        float val = input_matrix[offset + p];
        if (val > VALID_THRESHOLD && valid_count < MAX_BUFFER_SIZE) {
            buffer[valid_count++] = val;
        }
    }

    float result = EMPTY_VALUE;
    if (valid_count > 0) {
        sort(buffer, valid_count);

        int mid = valid_count / 2;
        result = buffer[mid];

        // even number of elements - average of two middle values
        if (valid_count % 2 == 0) {
            result = (buffer[mid - 1] + result) / 2.0f;
        }
    }

    output_medians[ts] = result;
}
```

Listing 6.2: reduce_patients_kernel for median across patients (kernel.cl)

Two functions are implemented for sorting - **Bubble Sort** and **Shell Sort**. Bubble Sort should be more suitable for a small number of patients (16), while Shell Sort is more efficient for larger datasets. The performance of both approaches is measured in the next chapter. Shell sort performs sorting directly in global memory without allocating additional memory, while Bubble Sort uses a local buffer.

```
void shell_sort_global(__global float* data, int start_idx, int count) {
    for (int gap = count / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < count; i += 1) {
            float temp = data[start_idx + i];
            int j;
            for (j = i; j >= gap && data[start_idx + j - gap] > temp; j -=
    gap) {
                data[start_idx + j] = data[start_idx + j - gap];
            }
            data[start_idx + j] = temp;
```

```
        }
    }
}

void sort(float* arr, int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                float temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Listing 6.3: Sorting: Bubble Sort and Shell Sort (kernel.cl)

6.3 Kernel 3: Downsampling (Shell Sort)

This kernel performs downsampling of time slots based on the output from the second kernel. It accepts the number of target time slots as a parameter and calculates the median from relevant original slots for each target slot.

- **Input:** Output from Kernel 2 (one median for each time slot).
- **Task:** Calculate median from K consecutive time slots and thus perform downsampling.
- **Implementation:** Since K can be large (thousands of values), registers cannot be used. The kernel utilizes **In-Place Shell Sort** directly in the graphics card's global memory.

```
__kernel void reduce_slots_kernel(
    __global float* input_slots,
    __global float* output_final,
    const int num_total_slots,
    const int num_wanted_slots
) {
    // 1. Data compression (moving valid values to start of section)
    int valid_count = 0;
    int write_ptr = start_idx;
    for (int read_ptr = start_idx; read_ptr < end_idx; ++read_ptr) {
        if (input_slots[read_ptr] > VALID_THRESHOLD) {
            if (read_ptr != write_ptr)
                input_slots[write_ptr] = input_slots[read_ptr];
            write_ptr++;
            valid_count++;
        }
    }

    // 2. Sorting directly in global memory (no allocation)
    if (valid_count > 0) {
        shell_sort_global(input_slots, start_idx, valid_count);
        // ... median selection ...
    }
}
```

Listing 6.4: Downsampling with Shell Sort

6.4 Solution for BVP Data (740 million records)

Specific problems associated with volume appeared during BVP data processing:

- **Index Overflow:** The number of elements ($5.7 \times 10^6 \times 16 \times 8 \approx 740 \times 10^6$) fits into a 32-bit `int`, but during index calculation (e.g., multiplication), intermediate result overflow can occur. 64-bit arithmetic (`long`) was strictly introduced in kernels for memory addressing.
- **Filtration:** BVP data can be negative. A constant `EMPTY_VALUE = -1e30` was introduced for padding to distinguish missing data from valid negative values.

Chapter 7

Measured Results and Evaluation

This chapter presents benchmark results for various datasets. Measurements were performed repeatedly, and results were validated by cross-checking (Sequential vs. Parallel vs. GPU) to ensure that results are identical (within defined tolerance).

7.1 Hardware Configuration

Measurement took place on the following configuration:

- **CPU:** M1 Pro (8 cores)
- **GPU:** M1 Pro (14 cores)
- **RAM:** 16GB unified memory
- **OS:** macOS Tahoe 26.2

7.2 Data Loading Times

Input for algorithms are CSV files. Measurement includes time needed for reading files from disk and their conversion into `flat_data` format. Data categories DEXCOM, HR, and BVP have different file sizes due to different sampling intervals.

Table 7.1: Loading times of input files (15 files for DEXCOM data type, 16 files for HR and BVP data types)

Data Type (sampling interval)	Average single file size	Loading Time [ms]
Dexcom (5 min)	139 KB	166 ms
HR (1 sec)	15.8 MB	18 435 ms (18s)
BVP (15 ms)	1.28 GB	1 568 830 ms (26m 9s)

7.3 Dexcom Data (Low Load)

- **Characteristics:** Sampling every 5 minutes (288 slots/day).
- **Size:** Small data, minimal memory demand.

Table 7.2: Results for Dexcom Data

Method	Time [ms]
CPU Sequential	5.45483 ms
CPU Parallel + SIMD	0.8895 ms
CPU Parallel (No Vectorization)	4.79196 ms
GPU (Full Pipeline)	59.51 ms

Evaluation: For small data, the overhead associated with data transfer to GPU (PCIe latency) and kernel launching is greater than the calculation itself. Here, the CPU Parallel version dominates thanks to vector instructions, effective cache usage, and absence of overhead.

7.4 HR Data (Medium Load)

- **Characteristics:** Sampling every 1 second (86 400 slots/day).
- **Size:** Medium-sized data.

Table 7.3: Results for HR Data

Method	Time [ms]
CPU Sequential	1544.08 ms
CPU Parallel + Vectorized	226.49 ms
CPU Parallel (No Vectorization)	1375.42 ms
GPU (Full Pipeline)	19.66 ms

Evaluation: With increasing data volume, GPU speed becomes more noticeable. Parallel CPU with SIMD instructions still offers good performance, but is likely limited by implementation which vectorizes only the first part of calculation (median across days). The GPU version benefits from parallelization across all parts of calculation.

Interestingly, the GPU runs for a shorter time here than with Dexcom data. The reason is that the GPU runs first on Dexcom data and subsequently on HR data. Overhead of OpenCL initialization and kernel JIT compilation counts towards the first run and is not needed for the second run.

7.5 BVP Data (High Load)

- **Characteristics:** Sampling every 15 ms (approx. 5 760 000 slots/day).
- **Size:** Large data, millions of records. Memory and cache limits manifest here.
- **Specifics:** Data contain negative values, requiring robust padding handling (`EMPTY_VALUE = -1e30`).

Table 7.4: Results for BVP Data

Method	Time [ms]
CPU Sequential	80440.40 ms
CPU Parallel + Vectorized	16506.80 ms
CPU Parallel (No Vectorization)	68084.00 ms
GPU (Full Pipeline)	1583.48 ms

Evaluation: This is the scenario for which GPU optimization was designed. GPU is more than 36x faster than sequential CPU and more than 7.5x faster than parallel CPU with vectorization. Large data fully utilize GPU parallel architecture and minimize overhead associated with data transfer thanks to the pipeline.

7.6 Analysis of GPU Calculation Phases

For detailed understanding of GPU performance, measurement was performed by individual steps (kernels). This allows identifying pipeline bottlenecks.

- **1 Kernel:** Launching only `median_kernel` (calculation of medians for each patient and time slot). Complete result matrix (time slots × patients) is transferred back to RAM.
- **2 Kernels:** Additionally launching `reduce_patients_kernel`. Patient reduction on GPU, only one median for each time slot is transferred to RAM.
- **3 Kernels (Full):** Complete pipeline including `reduce_slots_kernel` (downsampling). Only final downsampled results are transferred back to RAM.

Table 7.5: GPU times according to number of active kernels (BVP data)

GPU Pipeline Configuration	Time [ms]
Only Kernel 1 (Median per Patient)	1094.36
Kernel 1 + Kernel 2 (Reduce Patients)	534.659
Kernel 1 + 2 + 3 (Full Pipeline)	1024.54

Analysis: Time for "1 Kernel" (1094 ms) is double compared to "2 Kernels" (535 ms). This indicates a **Memory Bottleneck** in data transfer from GPU. Output of the first kernel is approx 368 MB of data (92 million values), which must be transferred over bus to RAM. When using two kernels, this data remains in GPU memory and only the result after 2nd kernel (approx 23 MB) is transferred. Time savings on data transfer here significantly exceed computation time of the second kernel.

Chapter 8

Analysis and Discussion of Results

The goal of this chapter is to evaluate the efficiency of implemented algorithms based on measured times. We focus on comparing absolute performance (scalability) and parallelization efficiency (Karp-Flatt metric).

8.1 Speedup and Efficiency

Speedup (S) is defined as the ratio of sequential algorithm time to parallel algorithm time: $S = T_{seq}/T_{par}$. From measured data, it is evident that speedup characteristics change dramatically with data size.

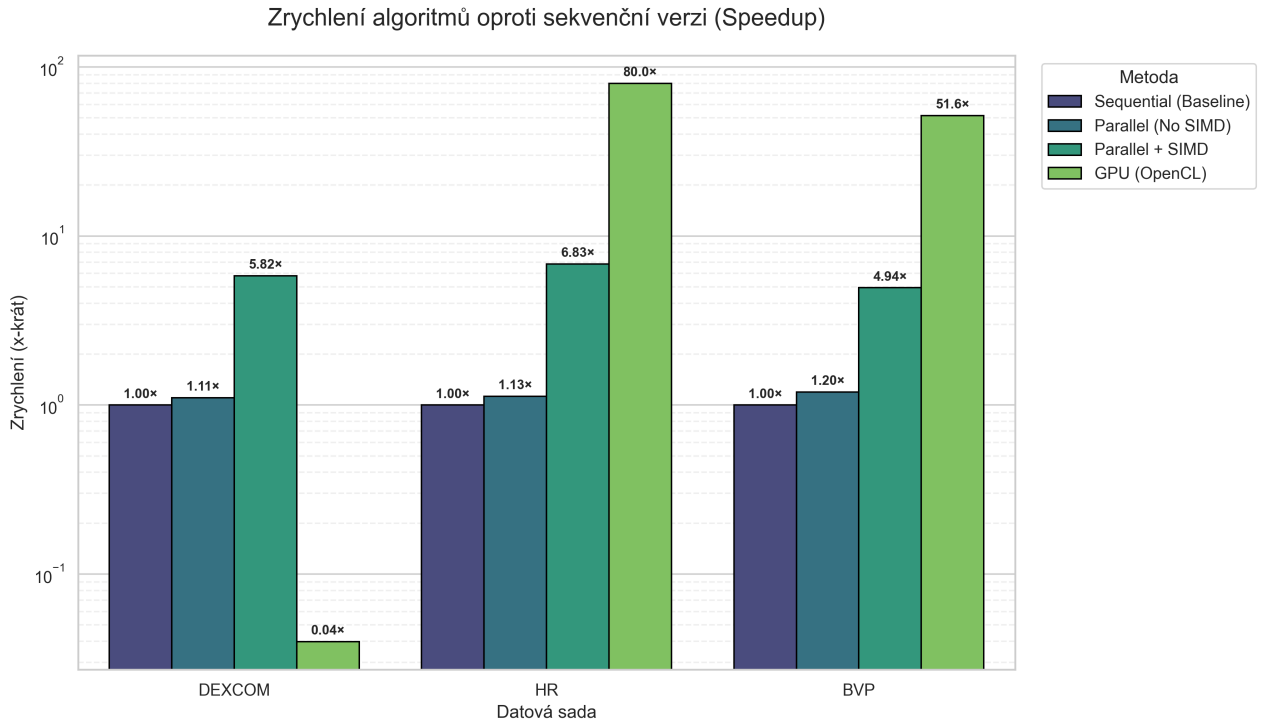

Figure 8.1: Graph of Speedup of individual methods compared to sequential version (Y axis is logarithmic). For small data (Dexcom), GPU is inefficient, while for large data (BVP), it dominates with speedup over 50×.

From graph 8.1, three key findings emerge:

1. **GPU Overhead on Small Data:** For the Dexcom set, GPU reaches speedup $S < 1$

(slowdown). Overhead associated with data transfer and kernel launch exceeds calculation itself here.

2. **GPU Dominance on Larger Data:** For HR and BVP sets, GPU is significantly more efficient. For BVP, GPU reaches speedup 51.6×.

3. **Role of SIMD Instructions:** In CPU implementation, the difference between pure parallelization (NoVect) and vectorization is visible. While 'Parallel (No SIMD)' reaches only 1.2× speedup for BVP, the optimized version 'Parallel + SIMD' reaches speedup 4.9×.

8.2 Karp-Flatt Metric

For deeper analysis of parallelization efficiency on CPU, the Karp-Flatt metric (e) was used, which estimates the serial portion of code including parallelization overhead. Calculation was performed for Apple M1 Pro processor with core count $p = 8$.

$$e = \frac{\frac{1}{S} - \frac{1}{p}}{1 - \frac{1}{p}}$$

Low value of e indicates effective parallelization. Values in Table 8.1 show how effectively the algorithm can utilize available cores.

Table 8.1: Karp-Flatt metric for CPU implementations ($p = 8$)

Dataset	**Method**	**Speedup (S)**	**Karp-Flatt (e)**	**Efficiency**
Dexcom	CPU Par + SIMD	5.82	0.053	High
HR	CPU Par + SIMD	6.83	0.024	Very high
BVP	CPU Par + SIMD	4.94	0.089	Medium
BVP	**CPU Par (No SIMD)**	**1.19**	**0.817**	**Critical**

Interpretation:

- For the **HR** set, the metric reaches value 0.024, meaning only 2.4 % of time constitutes serial part or overhead. Parallelization is almost ideal here.
- For the **BVP** set (with vectorization), the value rises to 0.089 (8.9 % overhead). This is likely caused by memory bus saturation (Memory Bound), where 8 cores compete for access to 1.2 GB of data.
- Method **BVP without SIMD** shows $e = 0.817$. This means 81.7 % of calculation behaves serially. Without SIMD instructions which process 4 values at once and effectively utilize registers, the CPU is overwhelmed by thread management overhead and waiting for data, degrading performance almost to the level of sequential code.

8.3 Scalability over Time

The last graph displays absolute runtime depending on input size.

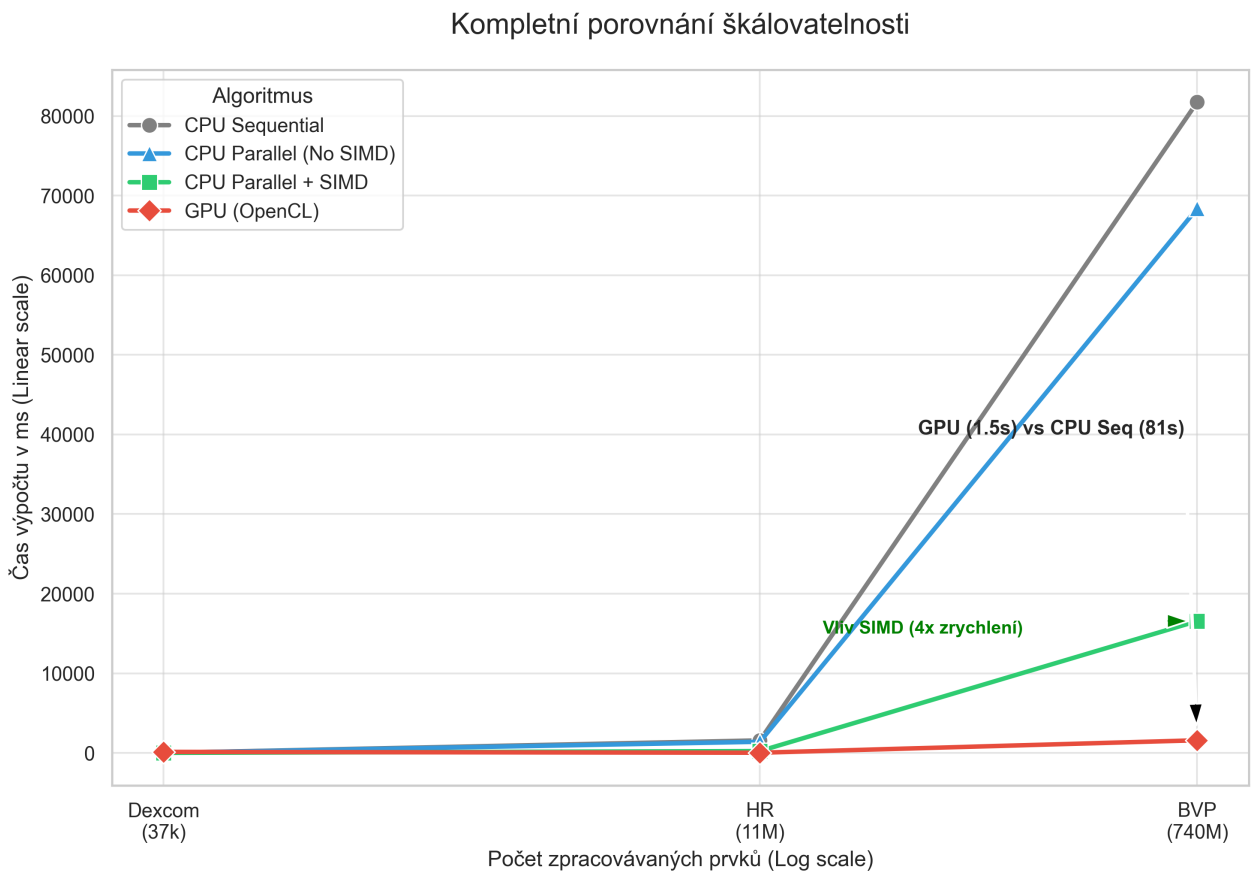

Figure 8.2: Dependency of calculation time on input data size (log-linear scale).

Graph 8.2 illustrates that while sequential and purely parallel solutions grow linearly with data size (steep increase for BVP), the GPU solution (red curve) maintains low processing time even for 740 million records. The green curve (SIMD) represents an ideal choice for medium-sized data, where it is faster than GPU (due to absence of data transfer), but for larger data it already loses to GPU.

Chapter 9

Discussion on Implementation

The project demonstrates effective processing of large datasets using various approaches - sequential CPU, parallel CPU with vectorization, and GPU acceleration. Each approach has its advantages and disadvantages depending on data size and available hardware. For larger datasets (BVP), GPU acceleration is clearly most efficient, while for smaller datasets (Dexcom), the optimized parallel version on CPU might be more advantageous due to lower overhead.

The work successfully solves several technical challenges, including effective use of SIMD instructions on CPU and implementation of robust pipeline on GPU. However, areas for further improvement exist:

- **Vectorization of reduction phase on CPU:** Current implementation uses SIMD only for calculation of median across days. Vectorization of other calculation parts (median across patients, median during downsampling) could further improve performance of parallel CPU version. However, efficient implementation of sorting dynamically long array using vector instructions is demanding. I attempted this (see chapter 5.4), but without success.
- **Dynamic strategy selection:** Development of an adaptive system that would automatically choose the most efficient processing approach (CPU vs. GPU) based on data size and available hardware.

Given the size of some datasets (BVP), loading data from disk takes considerable time (more than 26 minutes). For real applications, it would be appropriate to consider optimizations in this area, for example by parallel data loading.

Chapter 10

Conclusion

The project successfully implements processing of large medical datasets in 3 ways - sequential CPU, parallel CPU+SIMD, and GPU. Key technical challenges were solved during development:

- **Data Filtration:** Specifics of BVP signal (negative values) forced a change in logic for detecting empty places. Introduction of constant `EMPTY_VALUE = -1e30` and unification of conditions in all methods ensured result consistency.
- **Optimization:**
  - **GPU:** Utilization of Sorting Networks in the first kernel and in-place Shell Sort in the third kernel minimized memory requirements.

The resulting application demonstrates that for massive data (BVP), GPU acceleration is highly efficient, while for smaller datasets (Dexcom), the optimized parallel version on CPU is more suitable.

Appendix A: User Manual

The application is designed as a command line interface (CLI) tool, allowing flexible benchmark configuration. The user can choose which datasets to process, which calculation methods (sequential, parallel, GPU) to use, and where to save results. **Vector instructions NEON SIMD specific for ARM architecture are used in the application.**

Compilation and Build

The project uses the CMake build system. To build the application, it is necessary to have a compiler supporting C++23 and OpenCL library installed.

```
mkdir build
cd build
cmake ..
make
```

After successful build, an executable file `ppr_semestralka` is created in directory `build`. Command `./ppr_semestralka -h` displays help for controls.

Adding Data Folder

Input data should be placed in folder `data` in the project root directory (not in folder `build`). Structure of folder `data` should be following:

```
data/
    001/
        DEXCOM_001.csv
        HR_001.csv
        BVP_001.csv
    002/
        DEXCOM_002.csv
        HR_002.csv
        BVP_002.csv
    003/
        DEXCOM_003.csv
        HR_003.csv
        BVP_003.csv
    ...
```

Command Line Parameters

The program accepts the following switches for run configuration:

`-d, -datasets <list>` Specifies comma-separated list of datasets to process.

- Options: `dexcom`, `hr`, `bvp`
- Default: All (`dexcom,hr,bvp`)

`-m, -mode <list>` Specifies comma-separated list of calculation methods.

- Options: `seq` (Sequential), `par` (Parallel CPU), `gpu` (OpenCL), `all` (All)
- Default: `all`

`-k, -kernels <num>` Sets number of active phases (kernels) in GPU pipeline. Primarily serves for bottleneck analysis.

- `1`: Only calculation of medians for patient (massive data transfer).
- `2`: Added patient reduction on GPU (transfer minimization).
- `3`: Complete pipeline including downsampling (high computational load).
- Default: `3`

`-o, -output <file>` Path to file for logging benchmark results.

- Default: `benchmark_logs.txt`

`-h, -help` Prints help for controls.

Usage Examples

1. Basic Launch (all tests): Runs all algorithms over all datasets with default settings.

```
./ppr_semestralka
```

2. Benchmark only GPU on large data: Runs only GPU implementation over BVP data and saves results to specific file.

```
./ppr_semestralka -d bvp -m gpu -o gpu_results.txt
```

3. GPU Pipeline Analysis (Memory Bound test): Runs GPU calculation over BVP data only with first kernel to test memory throughput.

```
./ppr_semestralka -d bvp -m gpu -k 1
```

4. Comparison of CPU methods: Runs sequential and parallel version over HR and Dexcom data for verification of speedup on processor.

```
./ppr_semestralka -d hr,dexcom -m seq,par
```